1 Network size and synapse optimisation

1.1 Question

The target application is an always-on smart-home wake-word detector. A new one-second window of audio arrives every 250 ms, and the detector runs continuously. It must therefore (i) finish every inference within 250 ms, (ii) fit in the 144 KiB model memory of the RISC-V core, and (iii) use as little energy as possible while keeping its detection quality. This section asks how the size of the network trades energy per inference against detection quality within these limits. Size is counted in neurons and, more precisely, in synapses: the number of weighted connections and the number of bits each one stores.

In the deployed design every hidden neuron has a synapse to each of the 768 inputs. The $768H + 2H$ synapses hold 99.5 % of the model memory. On the plain RV32IM core each synapse costs one memory read and one multiply-add per inference. We therefore expect the synapses, more than the neurons, to set the cost.

Hypotheses (stated before the experiments).

H1 Within one hardware design, energy per inference is proportional to cycles. Synaptic multiply-adds dominate the RV32IM cycles, but not with the `kdot` accelerator.

H2 Quantisation-aware training keeps the validation F1 within the seed spread down to 4 bits per weight and fails at 2 bits.

H3 At an equal number of non-zero synapses, a large pruned network is at least as good as a small dense one.

H4 With current encoding the 12-step neuron update can be replaced by an exact table look-up; this saves at least half of the `kdot` cycles.

H5 With the deployed busy-polling firmware the energy per window does not depend on the network; savings require the core to idle between windows.

1.2 Method

Platform and network.

- **Hardware.** The network runs on a PicoRV32 (RV32IM, 100 MHz) in the programmable logic (PL) of the Zynq-7020, with 256 KiB of block RAM (BRAM).
- **Network.** It is a dense $768 \to H \to 2$ network of leaky integrate-and-fire neurons. The input layer computes the drive $d = \sum_i w_i x_i / 255 + b$ once (current encoding). Each hidden neuron then integrates d for 12 steps with a leak of $7/8$ and a threshold of 1024 (Q10 fixed point). The output scores are weighted spike counts.
- **Two hardware designs (fabrics).** The plain RV32IM core, and the same core with the `kdot` coprocessor, which streams int16 weights and uint8 inputs from BRAM at 0.75 cycles per multiply-add.
- **Training.** Every model is trained with the deployed recipe: Speech Commands v0.02 with hard near-miss variants, 35 epochs of which the last 10 are quantisation-aware, 3 seeds.

Quality. All quality numbers come from the integer reference model. The board reproduces it bit-exactly.

- **Choices** use the validation F1 of the clips and their hard near-miss variants only.
- **Reported** are the test F1 (11,005 clips) and the live-stream detection rate with the demo's "2 of 3" rule (212 keyword and 3,000 other test clips in background noise, 250 ms hop).

Latency and memory.

- **Cycles** are counted by the core's timer on the PYNQ-Z2. Each model runs 40 fixed inputs three times, and a result counts only if all scores and spike counts equal the integer reference. The mean $\bar{C}$ is used for energy and the maximum $C_{\max}$ for latency.
- **Memory** is the size of the model arrays in the format the firmware uses.

Energy.

- **Power estimate.** Power comes from Vivado 2026.1 `report_power` on the routed designs of both fabrics: vectorless, 25 °C, confidence "Medium" (Table 1). A USB power meter on the board input checked the method on a dual-core design of the group (two PicoRV32 cores, 132 BRAM). Loading that design raised the board power by 0.833 W; for the same design Vivado estimates 0.773 W of PL power, 7 % less. Losses in the board's voltage regulators are not part of the estimate.
- **Host excluded.** The ARM processing system (1.26 W) acts as the host and is excluded.
- **Power terms.** $P_{\mathrm{PL,dyn}}$ is the dynamic power of the PL. $P_{\mathrm{PL,stat}}$ is the static power of its supply rails. P_{core} is the share of the core with its BRAM, and $P_{\mathrm{infra}} = P_{\mathrm{PL,dyn}} - P_{\mathrm{core}}$ is the AXI interface.
- **Formulas.** With $f = 100\,\mathrm{MHz}$ and $T = 0.25\,\mathrm{s}$:

$$E_{\mathrm{inf}} = P_{\mathrm{PL,dyn}}\,\bar{C}/f \tag{1}$$

$$E_{\mathrm{win}}^{\mathrm{busy}} = (P_{\mathrm{PL,dyn}} + P_{\mathrm{PL,stat}})\,T \tag{2}$$

$$E_{\mathrm{win}}^{\mathrm{gated}} = (P_{\mathrm{PL,stat}} + P_{\mathrm{infra}})\,T + P_{\mathrm{core}}\,\bar{C}/f \tag{3}$$

$$E_{\mathrm{win}}^{\mathrm{scaled}} = P_{\mathrm{PL,stat}}\,T + P_{\mathrm{PL,dyn}} \cdot 1.1\,C_{\max}/f \tag{4}$$

- **Operating scenarios.** Equation (2) is the deployed firmware, which polls for work and never idles. Equation (3) stops the core clock between windows. Equation (4) lowers the PL clock until the worst-case inference just meets the deadline with 10 % slack (dynamic power proportional to frequency). Average power is E_{win}/T.
- **Why only cycles matter within a fabric.** The vectorless estimate does not depend on the workload. Within one fabric, energy ratios therefore equal cycle ratios, which are measured exactly.

Statistics and selection.

- Each configuration is trained with seeds 0, 1 and 2 and reported as mean ± sample standard deviation.
- **Selection rule (fixed before the experiments).** A design qualifies if its mean validation F1 is at least that of the best design minus that design's standard deviation. Among the qualifying designs within 250 ms and 144 KiB, the one with the lowest E_{inf} is recommended.

Table 1: Raw Vivado power estimate (W) of the two fabrics. Rows marked "Derived" are computed from the rows above. The block RAM is read on both ports in every cycle, so its power does not depend on the workload.

Quantity (W)	RV32IM	RV32IM + `kdot`
Total on-chip power	1.683	1.688
Dynamic power	1.536	1.540
Device static power	0.147	0.147
Dynamic: PS7	1.258	1.258
Dynamic: Clocks	0.011	0.011
Dynamic: Slice Logic	0.005	0.006
Dynamic: Signals	0.008	0.009
Dynamic: Block RAM	0.251	0.251
Dynamic: DSPs	0.003	0.006
Dynamic: I/O	0.000	0.000
Dynamic: core `u_soc` (CPU, `kdot`, BRAM)	0.270	0.274
of which PicoRV32 `u_cpu`	0.011	0.011
of which `kdot` `u_kdot`	–	0.005
Dynamic: AXI interface `u_ps_if`	0.002	0.002
Dynamic: AXI protocol converter `pc0`	0.005	0.005
Static, PL rails (Vccint, Vccaux, Vccbram, Vccadc, Vcco)	0.085	0.085
Static, PS rails (Vccp*, DDR, MIO)	0.062	0.062
Derived: $P_{\mathrm{PL,dyn}}$ = dynamic − PS7	0.278	0.282
Derived: P_{core} (`u_soc`)	0.270	0.274
Derived: $P_{\mathrm{infra}} = P_{\mathrm{PL,dyn}} - P_{\mathrm{core}}$	0.008	0.008

1.3 Results

1.3.1 Network size

- **Energy grows linearly with H** (Figure 1a, Table 2): 259 μJ per neuron with RV32IM and 11.4 μJ with `kdot` (linear fits for $H \geq 32$, $R^2 > 0.9999$). The deployed 95-neuron model uses 24.7 mJ and 1.10 mJ per inference.
- **Quality stops improving at about 16 neurons** (Figure 1b). Test F1 rises from 0.690 ± 0.074 at 8 neurons to 0.764 ± 0.036 at 16 neurons and then stays within the seed spread up to 95 neurons.
- **Above 16 neurons, extra neurons buy energy, not quality.** At 95 neurons a model costs 6 times the energy of a 16-neuron model.

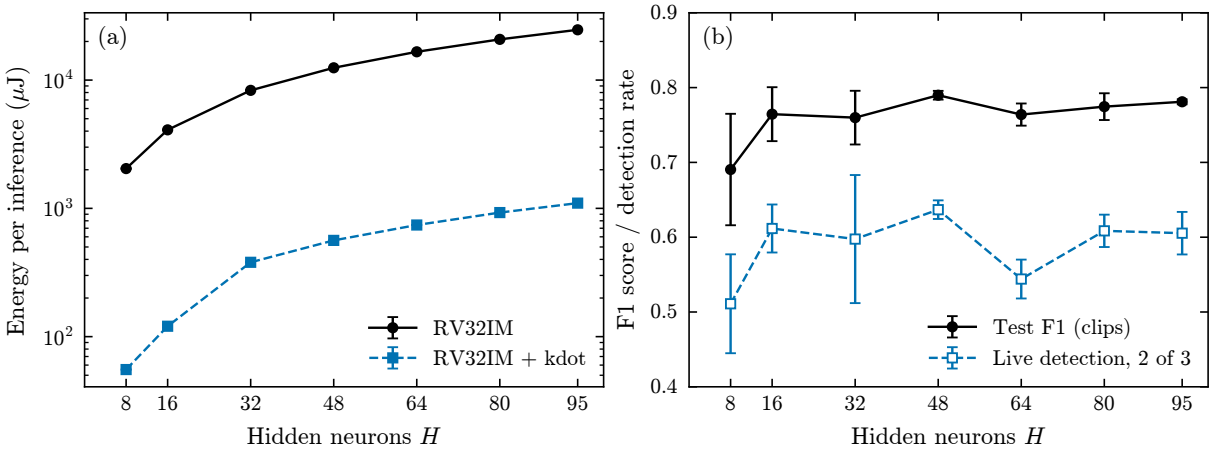


Figure 1: (a) Energy per inference (E_{inf}, Equation (1)) and (b) detection quality against the number of hidden neurons. Mean ± sample standard deviation over three seeds; the energy error bars are smaller than the markers.

Table 2: Network size: model memory, worst-case latency and energy per inference on both fabrics, and test quality (mean over three seeds).

		RV32IM		RV32IM + `kdot`			
H	Model (KiB)	$t_{\max}$ (ms)	$E_{\inf}$ (mJ)	$t_{\max}$ (ms)	$E_{\inf}$ (μJ)	Test F1	Live 2 of 3 (%)
8	12.0	7.37	2.04	0.22	55	0.690 ± 0.074	51.1
16	24.1	14.75	4.09	0.45	121	0.764 ± 0.036	61.2
32	48.3	29.95	8.32	1.38	380	0.760 ± 0.036	59.8
48	72.4	44.89	12.47	2.03	563	0.790 ± 0.006	63.7
64	96.5	59.80	16.62	2.66	742	0.764 ± 0.015	54.4
80	120.6	74.75	20.77	3.32	927	0.775 ± 0.018	60.9
95	143.2	88.76	24.66	3.94	1099	0.781 ± 0.003	60.5

Operating scenarios (Figure 2).

- **The deployed firmware makes the network irrelevant for energy.** It never idles, so the PL draws 363 mW (RV32IM) or 367 mW (`kdot`) for every network size.
- **The 256 KiB BRAM dominates.** It is 0.251 W of the 0.28 W of PL dynamic power.
- **Size only matters if the core idles between windows.** With clock gating the average power ranges from 101 to 189 mW for RV32IM, but only from 93.2 to 97.3 mW with `kdot`. There, the static power and the interface (93 mW) set the floor, and the network adds 0.2–4.4 mW.

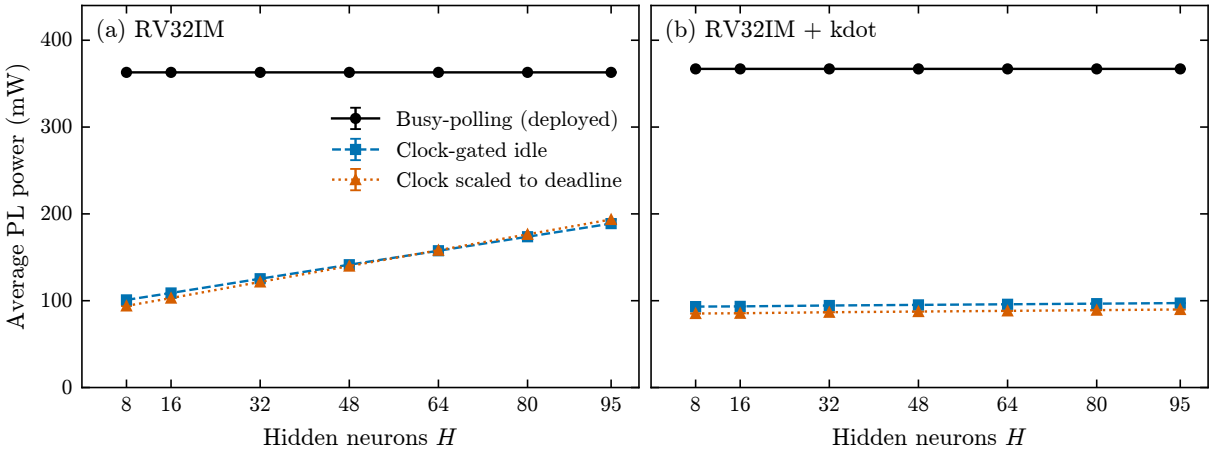


Figure 2: Average power of the programmable logic at one window per 250 ms, for the three operating scenarios of Equations (2) to (4): (a) RV32IM, (b) RV32IM with `kdot`.

1.3.2 Synapse precision

The input weights of the 16- and 48-neuron networks were trained with n-bit weights. One power-of-two scale per layer was fixed at the start of quantisation-aware training (QAT), using the same rule as Mark's study of the 128-input network. As a control, the trained 16-bit models were only rounded to n bits afterwards.

- **The deployed weights already use only 10 of their 16 bits.** The largest weight of the 21 size-study models is 274–497 in Q10 units.
- **Down to 4 bits the validation F1 is unchanged within the seed spread** (Figure 3a, Table 3). At 3 bits it drops slightly; at 2 bits the network fails, because almost every weight then rounds to zero.
- **The result transfers from Mark's network.** Relative to its own 8-bit model, our 768-input network behaves like Mark's 128-input network (Figure 3b). It degrades less at 3 bits (-0.005 to -0.009 test F1, against -0.045).

- **Training with the rounding is what makes low precision work.** Rounding a finished model loses more: at 48 neurons, 0.714 ± 0.014 at 4 bits and 0.44 ± 0.32 at 3 bits, against 0.742 and 0.732 with QAT.
- **Low precision prunes for free.** At 4 bits, 58 % of the 48-neuron network's weights are exactly zero (84 % at 3 bits).

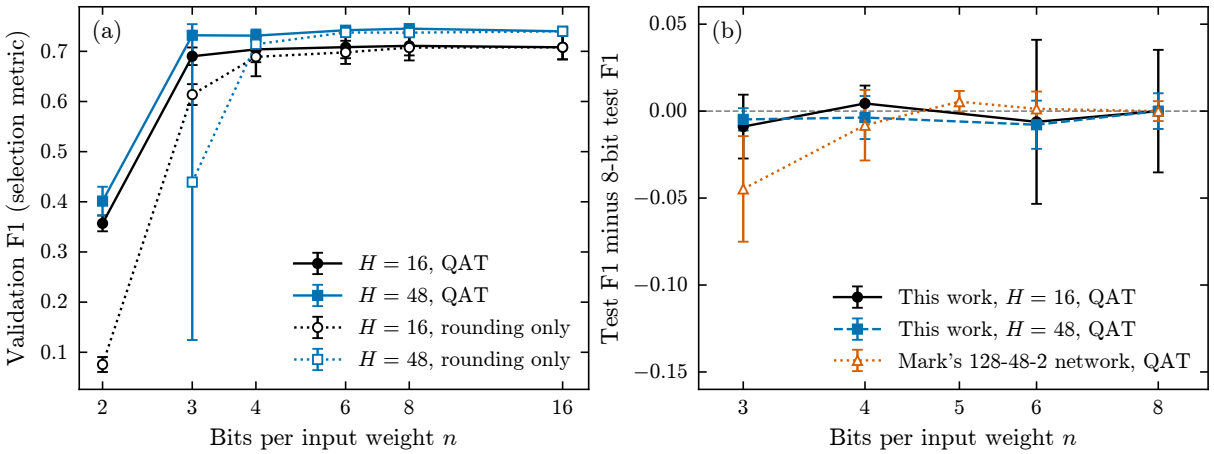


Figure 3: Synapse precision. (a) Validation F1 (selection metric) against the bits per input weight; QAT and rounding of the trained 16-bit models. (b) Change of the test F1 relative to each network's own 8-bit model, for this work and for Mark's 128-input network. The test sets differ, so only changes are compared. At 2 bits the change is -0.41 ($H = 16$), -0.40 ($H = 48$) and -0.55 (Mark), off scale.

- **Precision buys memory, not energy, on this core** (Figure 4). Packed 4-bit weights shrink the 48-neuron model from 72.4 to 18.4 KiB. The packed loop, however, needs extra instructions to extract each weight, and the PicoRV32 is limited by instruction fetch.
- **Even the 16-bit control loop is slower than the release loop:** 16.2 against 12.5 mJ per inference at 48 neurons. Within the packed loop, fewer bits mean more extraction work: 8 bits cost 14.1 mJ, 4 bits 18.3 mJ and 2 bits 28.1 mJ.
- **`kdot` cannot use packed weights.** It streams int16 weights, so a 4-bit model on `kdot` still occupies int16 memory.

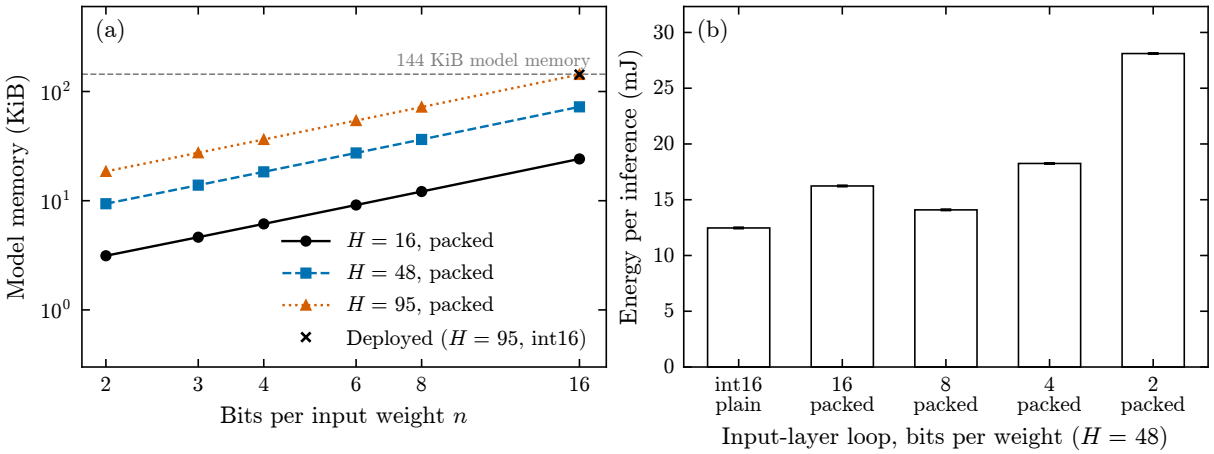


Figure 4: Cost of synapse precision. (a) Model memory with packed n-bit input weights. (b) Energy per inference on RV32IM at $H = 48$: the release int16 loop and the packed loop at 16, 8, 4 and 2 bits (measured on the board, bit-exact).

Table 3: Synapse precision: validation F1 (mean ± sd, three seeds) with QAT ($H = 16$, $H = 48$) and with rounding only ($H = 48$), share of zero weights, packed memory and measured energy of the packed loop at $H = 48$ (RV32IM). The 16-bit row is the size-study model with the release loop storage; its packed-loop energy is the control. n/a: no usable model (all weights round to zero).

n	$H = 16$	$H = 48$	$H = 48$, rounding only	Zeros (%)	Memory (KiB)	E_{inf} (mJ)
16	0.708 ± 0.024	0.740 ± 0.003	0.740 ± 0.003	1	72.4	16.2
8	0.711 ± 0.019	0.745 ± 0.005	0.737 ± 0.001	4	36.4	14.1
6	0.708 ± 0.022	0.742 ± 0.003	0.738 ± 0.003	16	27.4	–
4	0.704 ± 0.026	0.742 ± 0.008	0.714 ± 0.014	58	18.4	18.3
3	0.690 ± 0.018	0.732 ± 0.007	0.439 ± 0.315	84	13.9	–
2	0.357 ± 0.016	0.401 ± 0.029	n/a	100	9.4	28.1

1.3.3 The neuron update

- **The 12-step update has an exact closed form.** With current encoding the drive of a neuron is constant. Its 12-step spike count is therefore a non-decreasing staircase of the drive (verified for every drive in $\pm 2^{20}$): no spike below a drive of 161, twelve from 1024.
- **A table replaces it bit-exactly.** The firmware counts how many of the 12 thresholds the drive reaches, and the board results stay bit-exact.
- **The saving depends on the fabric** (Figure 5). On RV32IM the table saves 2–3 % of the cycles, since the synapses take about 97 %. With `kdot` the neuron update was 62–70 % of the cycles, and the table cuts the energy per inference from 120.5 to 45.0 µJ ($H = 16$), 562.7 to 165.4 µJ ($H = 48$) and 1098.6 to 323.0 µJ ($H = 95$).

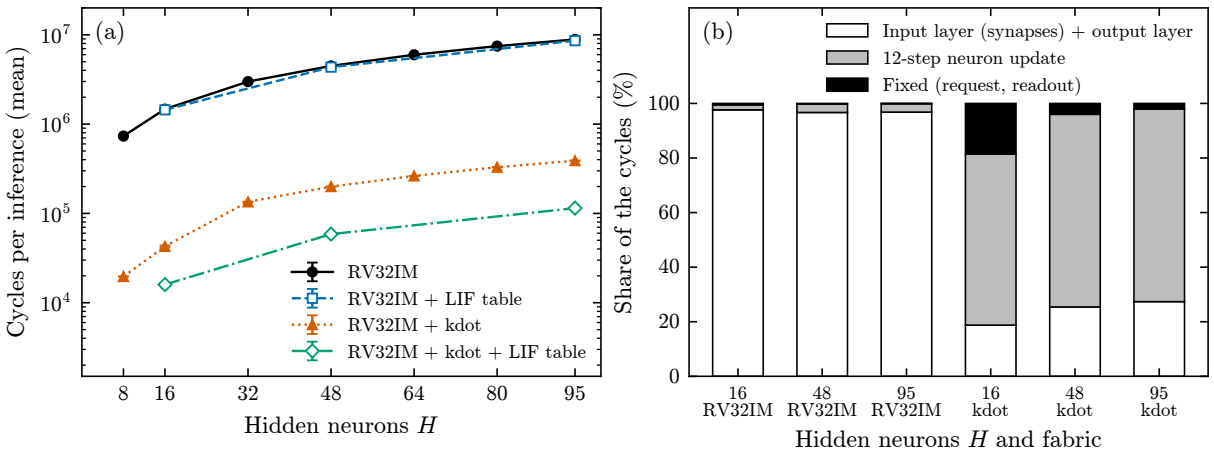


Figure 5: The neuron update. (a) Mean cycles per inference for the four firmware variants (all bit-exact on the board). (b) Share of the cycles: the 12-step neuron update is the part removed by the exact threshold table; the fixed part is the intercept of the size study's linear fit.

1.3.4 Synapse count

The 48- and 95-neuron networks were pruned to a fixed fan-in of K inputs per neuron. Pruning removed the smallest weights of every neuron gradually (epochs 3–20); the mask was then frozen for the remaining training, including the quantisation-aware epochs. With a constant K the firmware needs no row pointers, and its run time does not depend on the data. The sparse RV32IM loop stores one 32-bit word per synapse: the weight in the upper half and the input index in the lower bits.

- **A large pruned network beats a small dense one with the same number of synapses** (Figure 6a). With 6,144 non-zero synapses the dense 8-neuron network reaches a validation F1 of 0.643 ± 0.050; the 48-neuron network pruned to $K = 128$ reaches 0.727 ± 0.011. With 12,288 synapses it is 0.708 ± 0.024 dense ($H = 16$) against 0.733 ± 0.011 pruned ($H = 48$, $K = 256$).

- **Most synapses are redundant.** With $K = 64$, i.e. 3,072 synapses, 8 % of the dense 48-neuron network, the validation F1 is still 0.727 ± 0.008, against 0.740 ± 0.003.
- **On RV32IM energy follows the non-zero synapses** (Figure 6b). Each synapse costs about 141 cycles in the sparse loop, against about 120 for a dense multiply-add. The 48-neuron network at $K = 64$ needs 1.68 mJ per inference instead of 12.5 mJ, 7.4 times less.
- **`kdot` gains nothing from pruning.** It streams every weight, zero or not: the pruned 48-neuron model takes 200,021 cycles at most, the dense one 203,253.
- **Combining pruning and precision works.** 4-bit weights with $K = 256$ keep a validation F1 of 0.732 ± 0.005. They would need 21.4 KiB in a compact format (10-bit index plus 4-bit weight per synapse), against 72.4 KiB for the dense int16 model.
- **The 32-bit sparse format only saves memory below 50 % density.** The 95-neuron network at $K = 512$ (67 % density) does not fit the 144 KiB model memory in this format.

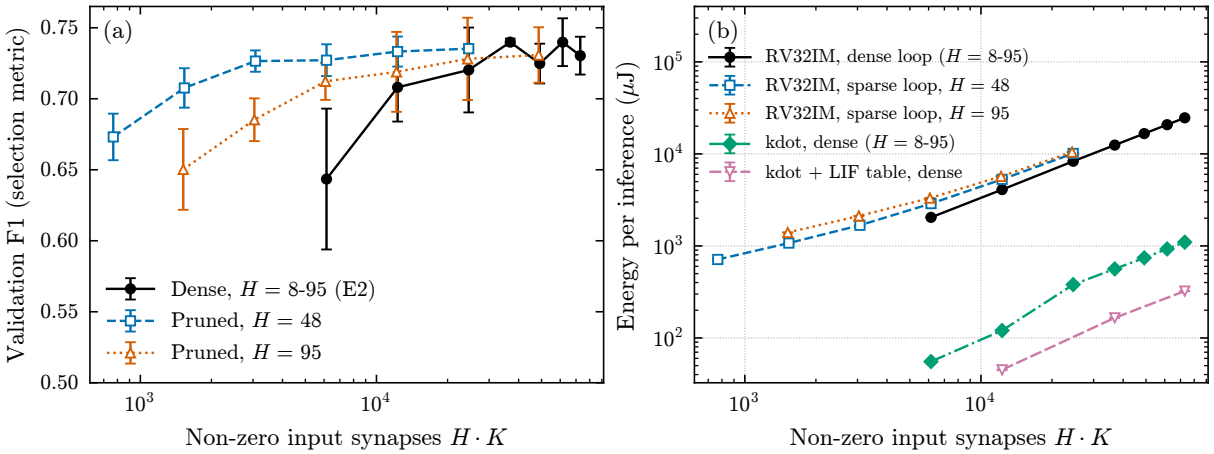


Figure 6: Synapse count. (a) Validation F1 against the number of non-zero input synapses for small dense networks and for 48- and 95-neuron networks pruned to a fixed fan-in K. (b) Energy per inference against the number of non-zero synapses for the dense and sparse RV32IM loops and the dense `kdot` firmware, without and with the exact neuron-update table.

1.3.5 A sparse accelerator

The results so far left the two best levers incompatible. Pruning saves energy only on the plain core, and `kdot` is fast only for dense weights. We therefore extended the coprocessor with two instructions; `kdot` itself is unchanged.

- `kxload` copies the 768 input bytes once per inference into a 1 KiB buffer inside the unit (distributed RAM).
- `ksdot` streams the kept synapses of one neuron in the 32-bit format of the sparse loop. For each synapse it looks up the input byte in the buffer and multiplies it with the weight in the existing `kdot` multiplier and accumulator, one synapse per cycle.

Verification.

- **Simulation.** A unit testbench in Vivado's simulator passed all 48 checks: the old `kdot`, the new instructions on random data and edge values, and back-to-back calls.
- **The new design meets timing at 100 MHz.** Worst slack is +0.545 ns. It uses 3,243 instead of 2,881 LUTs (220 of them as the buffer), and its estimated PL dynamic power is unchanged at 0.282 W.
- **The deployed firmware still runs identically on the new bitstream:** the same cycle counts and bit-exact results.

Results.

- **Every pruned model is bit-exact on the sparse accelerator.**
- **The cycle counts follow the prediction made before the hardware was built** (Figure 7a, Table 4). The prediction was that only the dot-product time changes, by $-H(576-K)+200$ cycles. All 16 configurations land within 170 cycles of it.
- **Pruning now pays off with the accelerator.** The 48-neuron network pruned to $K = 64$ needs 34,221 cycles and 96.5 μJ per inference instead of the deployed model's 58,667 cycles and 165 μJ (42 % less). Its model takes 12.4 instead of 72.4 KiB.
- **Below about $K = 64$ little is left to gain.** The cost approaches a floor of about 32,000 cycles: the input copy, the neuron table, the output layer and the request handling.
- **The price is quality.** The pruned 48-neuron models with $K \geq 64$ reach a validation F1 of 0.726–0.735 and live detection of 55–64 %, against 0.745 and 65.6 % for the dense model (Figure 7b). Smaller fan-ins lose more.
- **Weight precision makes no difference here.** Training the pruned models with 8-bit or 4-bit weights changes neither their quality nor their cost: each synapse is still a 32-bit word.

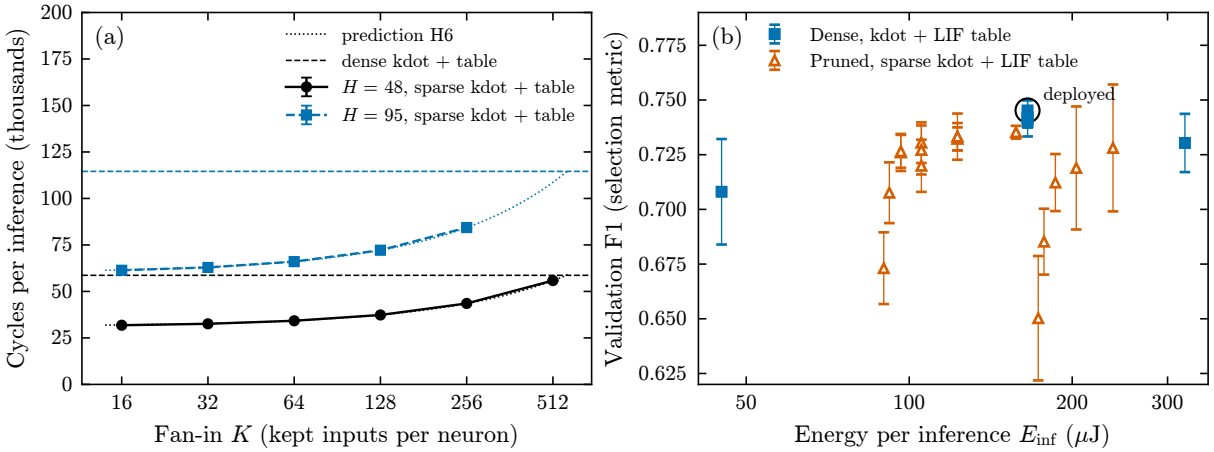


Figure 7: Sparse accelerator. (a) Measured cycles per inference against the fan-in K (mean over three seeds, all bit-exact), with the prediction stated before the hardware was built (dotted) and the dense `kdot` reference (dashed). (b) Validation F1 against energy per inference for the dense networks on `kdot` and the pruned networks on the sparse accelerator, both with the exact neuron-update table.

Table 4: Pruned networks on the sparse accelerator with the neuron-update table (mean over three seeds), next to the deployed dense model. ΔH6: measured minus predicted cycles.

Model	Cycles	Δ H6	E_{inf} (μJ)	Mem. (KiB)	Val F1	Test F1	Live (%)
deployed: $H = 48$, 8 bit, dense; `kdot` + table	58667	–	165	72.4	0.745 ± 0.005	0.789	65.6
$H = 48$, 16 bit, $K = 16$	31822	-165	90	3.4	0.673 ± 0.016	0.745	52.7
$H = 48$, 16 bit, $K = 32$	32587	-168	92	6.4	0.708 ± 0.014	0.761	54.7
$H = 48$, 16 bit, $K = 64$	34221	-70	97	12.4	0.727 ± 0.008	0.782	57.5
$H = 48$, 16 bit, $K = 128$	37341	-23	105	24.4	0.727 ± 0.011	0.784	56.9
$H = 48$, 16 bit, $K = 256$	43532	25	123	48.4	0.733 ± 0.011	0.780	59.9
$H = 48$, 16 bit, $K = 512$	55848	53	157	96.4	0.735 ± 0.003	0.773	64.2
$H = 48$, 8 bit, $K = 64$	34222	-69	97	12.4	0.726 ± 0.008	0.783	59.1
$H = 48$, 4 bit, $K = 128$	37343	-20	105	24.4	0.720 ± 0.012	0.787	58.8
$H = 48$, 8 bit, $K = 128$	37340	-23	105	24.4	0.730 ± 0.009	0.780	55.2
$H = 48$, 4 bit, $K = 256$	43537	30	123	48.4	0.732 ± 0.005	0.786	62.4
$H = 48$, 8 bit, $K = 256$	43530	23	123	48.4	0.733 ± 0.006	0.783	60.1
$H = 95$, 16 bit, $K = 16$	61381	-163	173	6.7	0.650 ± 0.028	0.700	50.8
$H = 95$, 16 bit, $K = 32$	62902	-162	177	12.6	0.685 ± 0.015	0.763	56.6
$H = 95$, 16 bit, $K = 64$	66025	-78	186	24.5	0.712 ± 0.013	0.789	60.4
$H = 95$, 16 bit, $K = 128$	72153	-30	203	48.3	0.719 ± 0.028	0.770	58.2
$H = 95$, 16 bit, $K = 256$	84352	9	238	95.8	0.728 ± 0.029	0.772	62.9

1.3.6 Design choice

- **The search covered 58 measured designs,** including the sparse accelerator. Figure 8 shows every combination of network and firmware that was measured bit-exactly on the board.
- **The Pareto front of validation F1 against energy** runs from the 16-neuron network on `kdot` with the table (45 μJ), through the pruned 48-neuron networks on the sparse accelerator (97–157 μJ), to the dense 48-neuron network on `kdot` with the table (165 μJ).
- **The rule selects 48 neurons with 8-bit weights on `kdot` with the table.** The best design has a validation F1 of 0.745 ± 0.005, so the qualifying threshold is 0.7405. Five designs qualify; no pruned network reaches the threshold.
- **The bit width does not change its energy.** Its 6- and 4-bit versions qualify as well, at the same energy: on `kdot` the weights are stored as int16 whatever their precision.
- **The sensitivity check now disagrees.** With the pooled standard deviation (0.019) the threshold is 0.7265. It selects the pruned 48-neuron network ($K = 64$) on the sparse accelerator, but that design passes the threshold by only 0.00006. We therefore keep the pre-registered choice, and report the sparse design as the low-energy alternative: 42 % less energy, for 0.019 lower validation F1 and 8 points less live detection.

Table 5 lists the designs on and around the Pareto front. Table 6 compares the choice, which has since been deployed, with the previous 95-neuron release.

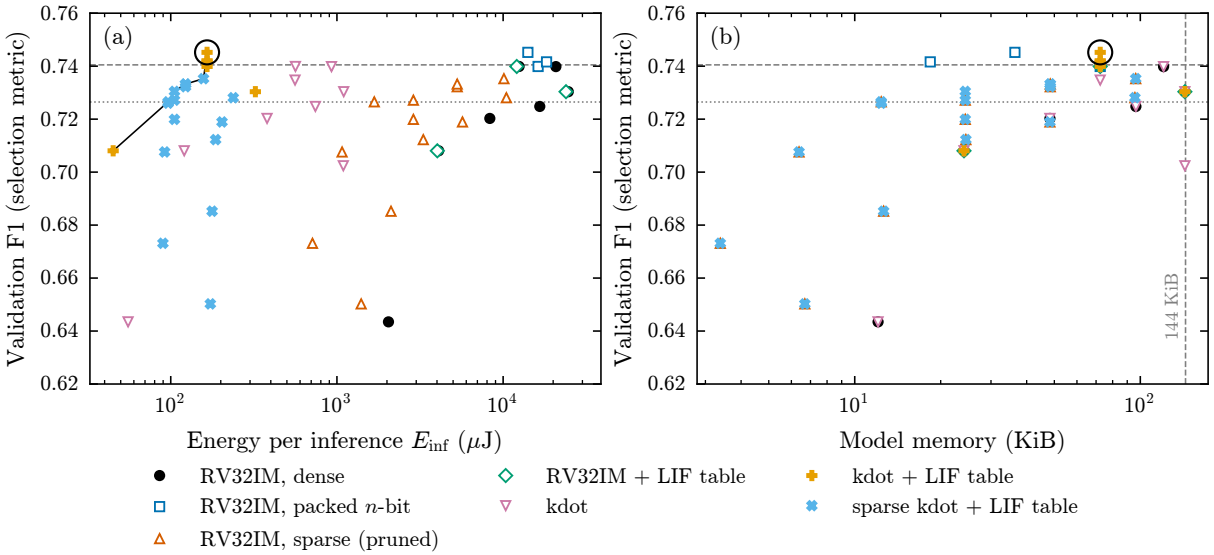


Figure 8: All measured designs (mean over three seeds). (a) Validation F1 against energy per inference, with the Pareto front (line). (b) Validation F1 against model memory. Dashed: the qualifying threshold of the selection rule; dotted: the threshold with the pooled standard deviation. Circle: the chosen design.

Table 5: Designs on and around the Pareto front (mean over three seeds). P_{gated} is the average PL power at one window per 250 ms with clock gating (Equation (3)). * chosen design, P Pareto front.

Design	Val F1	Test F1	Live (%)	Mem. (KiB)	$t_{\max}$ (ms)	E_{inf} (μJ)	P_{gated} (mW)
$H = 16$, 16 bit; `kdot` + table (P)	0.708 ± 0.024	0.764	61.2	24.1	0.16	45	93.2
$H = 48$, 16 bit, $K = 64$; sparse `kdot` + table (P)	0.727 ± 0.008	0.782	57.5	12.4	0.35	97	94.4
$H = 48$, 8 bit, $K = 128$; sparse `kdot` + table (P)	0.730 ± 0.009	0.780	55.2	24.4	0.38	105	94.4
$H = 48$, 8 bit, $K = 256$; sparse `kdot` + table (P)	0.733 ± 0.006	0.783	60.1	48.4	0.44	123	94.5
$H = 48$, 16 bit, $K = 256$; sparse `kdot` + table (P)	0.733 ± 0.011	0.780	59.9	48.4	0.44	123	94.5
$H = 48$, 16 bit, $K = 512$; sparse `kdot` + table (P)	0.735 ± 0.003	0.773	64.2	96.4	0.56	157	94.6
$H = 48$, 8 bit; `kdot` + table (*P)	0.745 ± 0.005	0.789	65.6	72.4	0.59	165	93.6
$H = 48$, 16 bit; `kdot`	0.740 ± 0.003	0.790	63.7	72.4	2.03	563	95.2
$H = 95$, 16 bit; `kdot`	0.730 ± 0.013	0.781	60.5	143.3	3.94	1099	97.3
$H = 48$, 16 bit, $K = 64$; RV32IM sparse	0.727 ± 0.008	0.782	57.5	12.4	6.06	1678	99.5
$H = 48$, 4 bit, $K = 256$; RV32IM sparse	0.732 ± 0.005	0.786	62.4	48.4	19.07	5293	113.6
$H = 48$, 16 bit; RV32IM	0.740 ± 0.003	0.790	63.7	72.4	44.89	12469	141.4
$H = 48$, 4 bit; RV32IM packed	0.742 ± 0.008	0.785	59.7	18.4	65.68	18252	163.9
$H = 95$, 16 bit; RV32IM	0.730 ± 0.013	0.781	60.5	143.3	88.76	24663	188.8

Table 6: The chosen design (mean ± sd over three seeds) against the deployed model, on the test data and the board.

Quantity	Chosen: $H = 48$, 8 bit; `kdot` + table	Previous: $H = 95$; `kdot`
Test precision	84.8 ± 2.0%	81.2%
Test recall	73.7 ± 1.2%	75.5%
Test F1	0.789 ± 0.010	0.782
Live “Sheila” detected, 2 of 3	65.6 ± 0.8%	60.9%
Live other words accepted, 2 of 3	0.36 ± 0.15%	0.33%
Near miss “shee” + other word	40.1 ± 4.8%	33.1%
Near miss other onset + “eila”	35.6 ± 1.4%	35.2%
Synthesized she- words, 2 of 3	33.3 ± 3.5%	31.5%
Model memory	72.4 KiB	143.3 KiB
Worst-case latency	0.59 ms	3.96 ms
Energy per inference	165 μJ	1103 μJ
Average PL power, busy-polling	367.0 mW	367.0 mW
Average PL power, clock-gated	93.6 mW	97.3 mW
Average PL power, clock scaled	85.7 mW	89.9 mW

Against the previous release (Table 6).

- **Cost.** The chosen design needs 165 μJ per inference instead of 1.10 mJ (6.7 times less), half the model memory, and 0.59 ms instead of 3.96 ms in the worst case.
- **Quality.** Its test F1 (0.789 ± 0.010) and live detection (65.6 ± 0.8 %) are at least as good, with the same rate of other words accepted.
- **The near-miss problem stays.** 40 % of the “shee” + other-word windows are accepted, against 33 %.
- **Average power changes little.** With clock gating it falls from 97.3 to 93.6 mW, because the static power and the interface set a floor of 93 mW. With the deployed busy-polling firmware it does not change at all.

1.4 Discussion

Hypotheses.

- **H1 is confirmed.** On RV32IM 97 % of the cycles are synaptic work; with `kdot` it is only 19–27 %.
- **H2 holds for quality, not for cost.** Four bits keep the quality and two bits fail. Fewer bits, however, increase the RV32IM energy instead of leaving it unchanged.
- **H3, H4 and H5 are confirmed.**
- **H6 is confirmed** (sparse accelerator: only the dot-product time changes), to within 170 cycles.

Which choice matters depends on the hardware. Network size, synapse count and synapse precision are not equally useful levers:

- **Precision** saves memory but no energy, because the core spends its time fetching and decoding instructions, not moving bits.
- **Pruning** is the effective synaptic lever on a plain RV32IM core.
- **With the `kdot` accelerator** synapses are cheap and the neuron update dominates. There, the exact table, a change in the algorithm rather than the network, gives the largest saving.

Pruning only paid off with the accelerator once the hardware could skip zeros. Adding that capability (`ksdot`) cut the energy per inference by up to 42 %; it cost 362 LUTs and no measurable power. This is the co-design argument in one step: the network's sparsity became useful only when the hardware was changed to exploit it. Packed low-bit weights would need the same kind of hardware support, a narrower synapse word; that was not built.

What the table shows about the network. The exact table also says something about the model. With current encoding the 12-step spiking dynamics reduce to a staircase function of the input with 13 levels. The network therefore behaves like a quantised artificial neural network and uses no timing information. This is consistent with the near-miss results: neither size, precision nor pruning changed how often words that share the onset of "Sheila" are accepted. The group's time-aware network with synaptic delays (Mateo) addresses exactly that; removing its delays cost 21 points of live recall in its own ablation for "yes".

Energy on this platform.

- **The network barely moves the average power on this FPGA.** The 256 KiB BRAM, read every cycle, and the static power dominate the programmable logic. A 6.7-fold energy saving per inference changes the always-on average power by only a few per cent, and not at all with the deployed firmware.
- **For a smart-home product the conclusions are different.** A dedicated chip would have little static power, and memory sized to the model. The per-inference energy and the model memory would then dominate, and they are what this study reduced.

Threats to validity.

- **Energy is estimated, not measured.** The values are vectorless Vivado estimates (confidence "Medium"). They were validated on the board only once: for a different design and at board level, where they were 7 % below the meter.
- **The power-analysed designs are rebuilds.** They are rebuilds of the release RTL with Vivado 2026.1, not the release bitstreams that produced the cycle counts.
- **Two of the scenarios are calculations.** Clock gating and clock scaling are not implemented.
- **Three seeds and a small number of positives.** Differences smaller than the seed spread are not interpreted. The test set has 212 positives.
- **The selection rule is fragile** when the best design has a small spread. With the sparse designs included, the pooled-sd check selects a different design, by a margin of 0.00006.
- **The sparse accelerator is an experimental bitstream.** Its power is a Vivado estimate like the others; it has not been deployed or tested with the microphone.
- **One implementation per format.** The packed and sparse loops are one implementation each; hand-optimised code could shift their cost.

1.5 Conclusion

For a keyword spotter on the PicoRV32 the size of the network is best counted in synapses, and their cost depends on the processor.

- **Quality saturates early.** It is flat from 16 hidden neurons on. A 48-neuron network keeps it with 8 % of its synapses or with 4-bit weights.
- **On plain RV32IM, pruning is the lever.** It cuts the energy per inference roughly in proportion to the synapses removed; lower precision only saves memory.
- **With `kdot`, the neuron update is the lever.** It takes most of the cycles, and an exact table removes 63–71 % of them.
- **The recommended design** is 48 neurons with 8-bit weights on `kdot` with the table. It needs 165 μJ and 0.59 ms per window, 6.7 times less energy than the deployed model, at the same quality and well inside the 250 ms and 144 KiB limits.
- **With hardware that skips zeros, pruning becomes a lever on the accelerator too.** The sparse accelerator cuts the energy per inference by another 42 % (96.5 μJ), at a measurable cost in quality. That makes it the low-energy point of the trade-off, not a free improvement.
- **On this FPGA the average power is set by the platform.** The memory and static power dominate, so an idle mode and right-sized memory are the next steps towards an always-on device.